OpenACC for MN-Core 仕様書

再配布禁止

綱島 隆太

令和 8 年 1 月 23 日

目次

1 概要 4
1.1 名称 4
1.2 対応バージョンと独自拡張 4
1.3 対応言語 5
1.4 想定ユースケース 5

2 デバイス固有の定義 6
2.1 オフロードの記述指示文 6
2.2 並列粒度階層の定義 6
2.3 ネストループの並列展開指示 7
2.4 同期・非同期実行 7

3 MN-Core 独自拡張 8
3.1 定義済みマクロ 8
3.1.1 __MNACC__ 8
3.1.2 __MNACF__ 8
3.2 ホスト・デバイス間通信 8
3.2.1 bcast 節 8
3.2.2 bcast_divide 節 9
3.3 loop ディレクティブの分割と基準となる配列の指示 9
3.4 配列分割指定 9
3.4.1 デフォルト分割 10
3.5 shadow 節及び reflect ディレクティブ 11
3.5.1 shadow 節 14
3.5.2 reflect ディレクティブ 15
3.6 集団通信（袖通信以外） 15
3.6.1 async_data ディレクティブ 16
3.6.2 alltoall ディレクティブ 16
3.6.3 その他 17
3.7 デバイス固有の指示の明示 17

4 OpenACC から要求される MNCL の仕様 18
4.1 袖交換の関数 18
4.2 ある PE のみで処理を変える場合の if 文 19
4.3 PE ローカル変数の使いまわし構文 20

5 **更新履歴** 23

1　概要

本文書では、目標となる仕様について簡潔に説明する。赤字のところは主に今後修正予定の箇所である。なお、現時点でここに記載した仕様のすべてが実装側でサポートされているとは限らない。リファレンス実装（コンパイラ）の仕様対応状況はコンパイラの README を参照。

なお、**設計上の想定使用状況や仕様の経緯、背景は設計書にまとめている**ので、詳しく知りたい場合はそちらも参照のこと。

また、まだ作成中だが、独自拡張以外の標準仕様も含めた OpenACC for MN-Core の具体的な使用方法についてはチュートリアルを参照のこと。標準仕様自体については OpenACC 公式の仕様書を参照。

コンパイラコマンドの使い方、詳細な実装の対応状況や制約についてはコンパイラの README を参照。

1.1　名称

基本的には、OpenACC for MN-Core または OpenACC/MN とする。
その他、MN-Core 向け OpenACC、MN-Core 対応 OpenACC、OpenACC/MN-Core と記載する。

1.2　対応バージョンと独自拡張

C 言語版、Fortran 版共通の仕様の概要について説明する。OpenACC/MN-Core は以下のような構成になっている。

- OpenACC 1.0 までの基本的な仕様
- OpenACC 2.0 以降の一部の分割コンパイルための仕様（enter/exit data、routine など）
- MN-Core 用の独自拡張

なお、必ずしも OpenACC のバージョン順通りにサポートしない可能性がある（例えば、2.5 の一部を実装していないが、2.6 の一部をサポートするなど）。

標準の OpenACC のインターフェイスによりサポートされる主な機能を以下に挙げる。

- ホスト・デバイス間データ転送
- デバイスメモリの確保と解放
- 既にデバイスに存在するデータの明示
- オフロード範囲の指定
- ループの並列分割階層（並列粒度）の指定
- リダクションの指示

独自拡張している箇所は主に以下の通りである。

- デバイス内の各階層のデータ分割指定
- ホスト・PE 間の集団通信の指示
- PE 間の集団通信の指示（特に袖通信）

1.3 対応言語

C、Fortran に対応する。実装においてパーサが未対応のため、C++ への対応は未定。ただし、対応する場合の仕様は C と同様。

1.4 想定ユースケース

MN-Core では DRAM が存在するが、PE から離れた位置にある。また、OpenACC/MN 開発開始当初、プロジェクトであるポスト富岳 FS では PE 毎にローカルな DRAM が存在するハードウェアを想定していた。以上より、ひとまず PE 上のメモリに収まるサイズのみを想定する。言い換えると、ハードウェアを完全な分散メモリ型として考える。

また、汎用 HPC 処理が可能な仕様とする予定であるが、ターゲットコードとしてまずはステンシル計算のコードを想定する。

2　デバイス固有の定義

以下では、標準仕様でデバイス固有（言語処理系実装依存）とされている部分について定義する。

2.1　オフロードの記述指示文

OpenACC の計算オフロード指示文には parallel、kernels、serial の 3 つがある。このうち、serial は parallel の並列度が 1 のときのエイリアスである。これに加えて、上記と loop を組み合わせた parallel loop などの書き方が存在する。

ただし、kernels は MN-Core の SIMD アーキテクチャに合致しない。GPU では kernels と parallel は実使用上ほぼ違いがなく、parallel も kernels もあまり区別なく使われている傾向にあるが、kernels はスレッドを前提とした設計となっており、ループごとに異なる並列度を指示することができてしまう。

したがって、OpenACC/MN では parallel で記述することを想定する。ただし、将来的には GPU コードの移植時を想定して kernels を用いている場合にも内部的に parallel と解釈することで変換自体は受け付けるようにする。serial についても念の為対応する。

なお、parallel ディレクティブでは標準仕様に沿って、ループに依存性が存在しないことを前提にコンパイルするため、並列実行した際の結果の保証はプログラマの責任とされる。parallel ディレクティブで independent 節が不要なのは MN-Core 向けでも共通である。kernels の場合は independent 節がないと保守的に parallel へ変換するので、直接 parallel で書くことを推奨する。

2.2　並列粒度階層の定義

OpenACC ではループに指定される分割時の並列粒度割当てについては，デバイスごとに解釈を定める必要がある．OpenACC/MN では以下のように解釈する．

- gang：PE レベル並列
- worker：割当てなし（記述があっても無視する）
- vector：ベクトル命令並列（FP64 の場合、最大長さ 4）

worker に何も割り当てないのは、適切な階層が無いことに加えて、PGI 及び NVHPC の OpenACC で worker を基本的に使用しない慣例を参考にしている。

gang 節の指定された loop ディレクティブの中に vector 節を指定した loop ディレクティブが存在することは問題ないが、逆は許容されない。なお、実装ではまだ未対応だが、gang と vector は一つの loop ディレクティブに併記することができる。

2.3 ネストループの並列展開指示

ネストループを並列展開するには必ず collapse 節を指定する必要がある。GPU 版 OpenACC のように省略して記述した場合、適切な並列展開はされない。また、collapse 節を使わずに、ループそれぞれに並列粒度 (gang、worker、vector）を指示しても、collapse 節を指定したときと同様の変換を行うことはできない。MN-Core では基本的に PE 並列で、あとはベクトル命令による実行ができるかどうかだけなので、gang レベルで一通りまとめて並列展開させないといけない。

実装ではまだ未対応だが、collapse 節を書くディレクティブに gang と vector を併記することで、ネストループ全体を PE 並列に展開したうえでベクトル命令の適用を試みることが可能である。

2.4 同期・非同期実行

まず、OpenACC における同期とは、ホストとデバイス間の同期と、並列実行単位の同期の両方がある。前者は async 節を指定した時点で非同期となり、後者を非同期にする際には async 節の引数に id を指定する。

OpenACC 標準でのデフォルト動作は同期実行である。これは OpenACC/MN-Core でも同じである。

非同期実行についてであるが、前者の意味において非同期実行をさせることは MN-Core でも可能である。ただし、後者の意味においては、MN-Core 自体が SIMD 動作であることから実際の動作としては同期実行しかできない。

async 節を記述したときの動作は、前者についてはホストと非同期実行する（まだ実装未対応)。後者についてまだ未定である。エミュレーションをするのであれば想定される動作としては、実質的な Atomic 動作で if 文同様に順番に実行していき、最後に同期を取る形になると思われる。id が付けられるので単純な Atomic ではなく Switch 文のような動作になるわけだが、ハードウェア上あり得ない動作であり、性能も見込めないため、混乱を避けるために後者については禁止（無視）にすることも検討している。

3　MN-Core 独自拡張

MN-Core 向けに拡張する部分は、主に MN-Core が分散メモリ型アーキテクチャであることによる拡張である。

ここから記述について説明するが、C 言語と Fortran それぞれの場合を示す。

3.1　定義済みマクロ

3.1.1　__MNACC__

C 言語版 OpenACC/MN コンパイラでは、`__MNACC__` が自動的に定義される。

3.1.2　__MNACF__

Fortran 版 OpenACC/MN コンパイラでは、`__MNACF__` が自動的に定義される。

3.2　ホスト・デバイス間通信

- スカラー変数はデフォルトでは全 PE へ放送される。
- 配列はデフォルトでは分割数の指示に従って分配される。

現時点では、ソースコード 1、ソースコード 2 のような記述を想定している。

ソースコード 1　その他の MN-Core 拡張（C 言語）

```
#pragma acc enter data copyin(a[:N][:N],b[:N],c[:N]) create(d[:N]) \
bcast(c, d) bcast_divide(b)
```

ソースコード 2　その他の MN-Core 拡張（Fortran）

```
!$acc enter data copyin(a(:N, :N),b(:N),c(:N)) create(d(:N)) &
!$acc bcast(c, d) bcast_divide(b)
```

3.2.1　bcast 節

bcast 節はホストから来る配列の放送を指示するための節である。引数は配列名であり、カンマで区切って複数の配列を指定する記述が可能である。copy、copyin、create の各節に放送したい配列が記述されているディレクティブに一緒に記述する。なお、ソースコード 1 及びソースコード 2 中の `d` は create 節で指定されているため、実際には `d` のための放送通信は発生しないが、すべての PE に同じ要素数が確保されることをコンパイラに知らせるために bcast 節に記述する。

3.2.2 bcast_divide 節

bcast_divide 節は loop ディレクティブの指示されたループにおいて計算自体は分割するが、すべての配列要素を各 PE に重複保持したい場合に指示する節である。引数は配列名であり、カンマで区切って複数の配列を指定する記述が可能である。

このような指示が必要なケースとして、計算が進む毎に PE 間にデータの依存性が発生する場合を想定している。共有メモリ型では PE 間のデータを互いに参照することが可能であるが、MN-Core は分散メモリ型であるため、事前に必要なデータを同期しておく必要がある。この節で指定された配列は放送されて各 PE に重複して確保されるものの、各 PE で計算される index 範囲は決まっており、各 PE 上の分割対象のループではそれぞれの担当範囲のみが計算される。同期については各 PE 間のデータを Allgather の要領で同期する async_data ディレクティブを合わせて用いる。

なお、この節で指定された配列は、ハードウェア上ではデータ自体が放送されている。そのため、loop ディレクティブの対象範囲外のコードで各 PE の担当範囲でない index にアクセスすることも可能である。ただし、async_data ディレクティブで同期する際に上書きされることから、各 PE の担当範囲でない index への書き込みはバグの原因となりうるため非推奨である。

3.3 loop ディレクティブの分割と基準となる配列の指示

gang レベルでのループの分割はあくまで、データの配置を参考に分割する。

例えば、ループの始点と終点がデータサイズと一致せず、index 上の始点と終点付近にあたるデータを計算しない場合（アプリ上、境界条件は計算しない等）もありうるが、コンパイラはあくまでループは現実装ではその計算オフロードブロックに指示されている present 節の一番先頭に記述されている配列の分割配置に基づいて分割をする。つまり、ループの回転数を基準に分割することはない。ループ自体の分割に関するチェックは、あくまで参照する配列の index が配列サイズからはみ出たり、分割した際に PE 上に存在しないデータを参照していないかの確認に留める。

なお、上記に基づき、現実装では袖の付いた配列がループの中にある場合、そのうちのループ回転数に対応するサイズの配列のどれかを必ず present 節の一番先頭に記述する必要がある。標準の OpenACC には present 節の順番に特に制約は無いため、MN-Core 用に順番を入れ替えても可搬性に悪影響はない。また、この仕様ではループごとに異なる分割を行えないが、標準の OpenACC でも parallel ディレクティブ下では常に一定の分割によって並列実行される仕様である。この present 節の拡張解釈を今後独自のディレクティブへ移行するかどうかは未定である。

3.4 配列分割指定

現実装では拡張ディレクティブには未対応だが、何も書かない場合のデフォルト分割には対応。**なお、当面はブロック分割のみとする。**

階層ごとに図 1 や図 2 のように分割数を定義する。分割は図 3 のように行われる。

階層ごとに、L2 は 1 パッケージあたりの L2BM の、L1 は一つの L2B 内部の L1BM の、mab は一つの L1B 内部の MAB の、pe は一つの MAB 内部の PE の分割数をそれぞれ示している。例えば L2 であれば、全 L2BM を用いてどのように分割するかということを記述する。

L2 などに続く丸括弧の中には配列名とその配列の各次元の分割数を記述する。a, b, c はそれぞれ配列名である。それぞれの次元の分割数は C 言語の場合、角括弧 [ ] の中に 0 以外の自然数で指定できる。Fortran の場合、配列名に続く丸括弧 ( ) の中に各次元ごとにカンマ , で区切って、0 以外の自然数で指定できる。ただし、分割数の値は各次元の分割数を掛けた値がその階層の個数の合計値を超えない範囲で指示する必要がある。

各配列の記述はセミコロン (;) で区切る。ただし、同じ分割を行う配列は、配列名の直後にコロンで区切って続けて書く省略記述が可能である。

なお、ここで設定するのは分割後のサイズではなく分割数であるため、配列サイズを変化させる場合でも、各次元が元のサイズの倍数で変化する場合（形が変わらない場合）には書き換え不要である。

```
1 #pragma acc enter data copyin(a[0:N][0:N],b[0:N][0:N],c[0:N]) \
2     L2(a,b[4][4];c[16]) L1(a,b[4][2];c[8]) \
3     mab(a,b[2][8];c[16]) pe(a,b[4][1];c[4])
```

図 1　分割数指定のための拡張節（C 言語の場合）

```
1 !$acc enter data copyin(a(1:N, 1:N),b(1:N, 1:N),c(1:N)) &
2     !$acc& L2(a,b(4, 4);c(16)) L1(a,b(4, 2);c(8)) &
3     !$acc& mab(a,b(2, 8);c(16)) pe(a,b(4, 1);c(4))
```

図 2　分割数指定のための拡張節（Fortran の場合）

3.4.1　デフォルト分割

データ分割のための拡張ディレクティブ（節）を省略した場合は、無印 MN-Core では正方形や立方体に最適化された以下の分割を行う。

- 2 次元では L2B, L1B, PE を [4][4], [4][2], [(4) *2][8] で分割
- 3 次元では L2B, L1B, PE を [4][4][1], [1][1][8], [4][4][(4)] で分割
- 1 次元では MAB の中の PE0 と PE1 とかで交互に（インターリーブで）配列を配置するようにする

2、3 次元のうち、(4) は同じ MAB に属することを示している。また、上記を合算すると 2 次元では [128][64]、3 次元では [16][16][32] の格子に分割され、それぞれの次元の格子数で割った値が

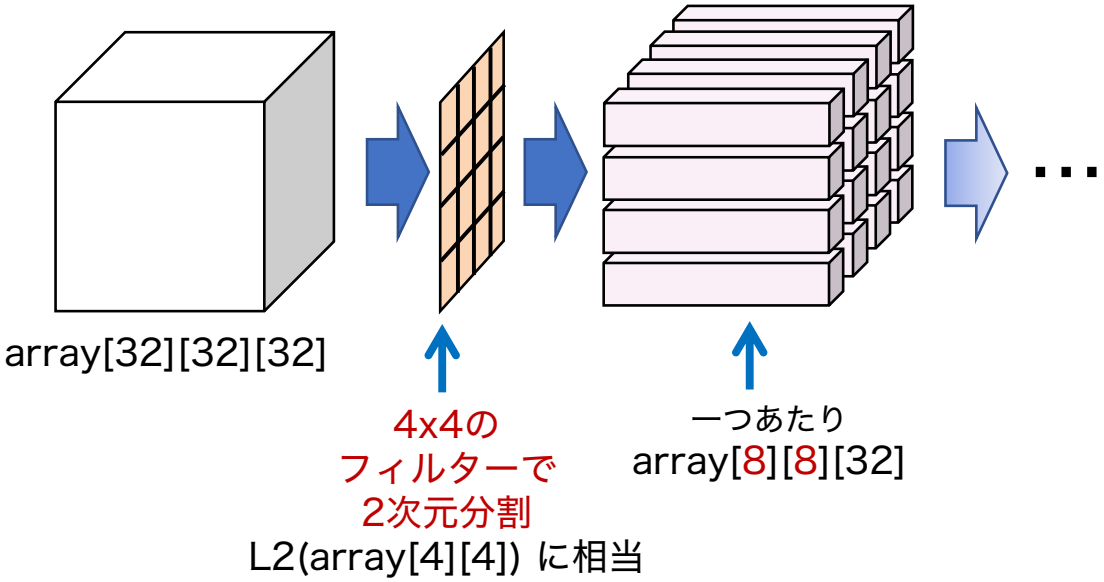

図 3　例：L2(array[4][4]) という記述があった場合の分割イメージ

1PE あたりの担当要素数となる。逆に言えば、この値未満の 2、3 次元配列では 1 要素も分配されない PE が発生し、全 PE を使い切らない。

基本的に正方形や立方体の形であれば、上記のデータ分割方法を指定することで通信効率を上げられることが MNCL 側で示されているため、OpenACC でもこれをデフォルトとする（**なお、これは無印 MN-Core の場合の話であり、デバイスの階層と PE 数によって変わる**）。正方形や立方体から大きく崩れなければ、通信効率は高く保たれる。

また、1 次元配列をこのようにするのは、こうしないと、MAB 単位の放送のバンド幅を有効活用するために、一旦境界の PE からその他 3 つの PE にデータを分散して、更に受け取った MAB 側でもそのデータをもう一回境界の PE に集約する必要があり、データ転送が 2 回余計に増えてしまうためである。

3.5　shadow 節及び reflect ディレクティブ

これらはステンシル計算を行う際に用いる。ステンシル計算のコード例を図 4、図 5 に示す。これらは XcalableMP、HPFortran を参考に、標準の OpenACC と整合性の取れたインターフェイスとする。

```
1  // Host to PE (PE 配置はデフォルトでいいなら省略可)
2      // 各次元の袖領域をshadow 節で指定 [マイナスの要素数:プラスの要素数]
3  #pragma acc enter data create(temp[0:128][0:128][0:128])
       copyin(a[0:128][0:128][0:128]) shadow(a[1:1][1:1][1:1])
4  // オフロード範囲の指定(parallel ディレクティブ)
5      // ループ分割はpresent 節の最初に指定された配列に従う
6      // 袖を持つ配列がある場合はそれを最初に指定
7      // 分割コンパイル対応でshadow 節をここでも指定
8  #pragma acc parallel present(a[0:128][0:128][0:128],
       temp[0:128][0:128][0:128]) shadow(a[1:1][1:1][1:1])
9  for(count=0; count<N; count++) {
10 #pragma acc loop gang collapse(3)
11     for(int i=1; i<128-1; i++){
12         for(int j=1; j<128-1; j++){
13             for(int k=1; k<128-1; k++){
14                 temp[i][j][k] =
15                     c1*(a[i-1][j][k]+a[i+1][j][k]
16                     +a[i][j-1][k]+a[i][j+1][k]
17                     +a[i][j][k-1]+a[i][j][k+1])
18                     +c2*a[i][j][k];
19     } } }
20 #pragma acc loop gang collapse(3)
21     for(int i=1; i<128-1; i++){
22         for(int j=1; j<128-1; j++){
23             for(int k=1; k<128-1; k++){
24                 a[i][j][k] = temp[i][j][k];
25     } } }
26 // reflect で袖交換の発生をコンパイラに知らせる
27 #pragma acc reflect(a)
28 } // count の for 文(=オフロード適用範囲)の終了
29 // PE to Host
30 #pragma acc exit data copyout(a[0:128][0:128][0:128])
       delete(temp[0:128][0:128][0:128])
```

図 4　ステンシル計算の例（C 言語）

```
1  ! Host to PE（PE 配置はデフォルトでいいなら省略可）
2      ! 各次元の袖領域を shadow 節で指定（マイナスの要素数:プラスの要素数）
3  !$acc enter data create(temp(1:128, 1:128, 1:128)) copyin(a(1:128, 1:128,
       1:128)) shadow(a(1:1, 1:1, 1:1))
4  ! オフロード範囲の指定（parallel ディレクティブ）
5      ! ループ分割は present 節の最初に指定された配列に従う
6      ! 袖を持つ配列がある場合はそれを最初に指定
7      ! 分割コンパイル対応で shadow 節をここでも指定
8  !$acc parallel present(a(1:128, 1:128, 1:128), temp(1:128, 1:128,
       1:128)) shadow(a(1:1, 1:1, 1:1))
9  do count=1, N
10 !$acc loop gang collapse(3)
11     do i=2, 128-1
12         do j=2, 128-1
13             do k=2, 128-1
14                 temp(i,j,k) = &
15                     c1*(a(i-1,j,k)+a(i+1,j,k) &
16                     +a(i,j-1,k)+a(i,j+1,k) &
17                     +a(i,j,k-1)+a(i,j,k+1)) &
18                     +c2*a(i,j,k)
19             end do
20         end do
21     end do
22 !$acc loop gang collapse(3)
23     do i=2, 128-1
24         do j=2, 128-1
25             do k=2, 128-1
26                 a(i,j,k) = temp(i,j,k)
27             end do
28         end do
29     end do
30 ! reflect で袖交換の発生をコンパイラに知らせる
31 !$acc reflect(a)
32 end do
33 !$acc end parallel ! Fortran では必要
34 ! PE to Host
35 !$acc exit data copyout(a(1:128, 1:128, 1:128)) delete(temp(1:128, 1:128,
       1:128))
```

図 5　ステンシル計算の例（Fortran）

3.5.1 shadow 節

ソースコード 3　C 言語の場合

```
shadow(配列名 [先頭の袖の幅:末尾の袖の幅] [多次元の場合は同様の記法で続ける]...，別の配列の記述(あれば) ...)

// 袖の幅が先頭と末尾で異なる例
shadow(array[2:1])

// 袖の幅が先頭と末尾で同じ場合(省略記法)
shadow(array[2])
```

ソースコード 4　Fortran の場合

```
shadow(配列名 (先頭の袖の幅:末尾の袖の幅，多次元の場合は同様の記法で続ける)...，別の配列の記述(あれば) ...)

! 袖の幅が先頭と末尾で異なる例
shadow(array(2:1))

! 袖の幅が先頭と末尾で同じ場合 (省略記法)
shadow(array(2))
```

shadow 節は配列の袖領域を指定するための節である。shadow に続き丸括弧の中に配列名、それに続いてそれぞれの次元の袖の幅を角括弧（Fortran の場合は丸括弧）の中に続ける。なお、C 言語の場合は次元ごとに角括弧を記述するが、Fortran では同じ丸括弧の中に次元ごとにカンマで区切って記述する。袖の幅は 0 以上の自然数で記述する。なお、先頭の袖の幅と末尾の袖の幅が同じ場合、省略記述が可能である。そうでない場合、先頭の袖の幅と末尾の袖の幅をコロン : で区切って記述する。複数の配列を指定する場合はカンマ , で区切って、別の配列の記述を続ける。

shadow 節は copy, copyin, create, present の節が指定されるディレクティブに一緒に指定する。

ファイルを跨いで有効になることはない（ファイル内限定のスコープ）。enter data ディレクティブを使う場合、そのタイミングで PE にデータが配置され、メモリ確保されるので、例え enter data ディレクティブを書くファイルに reflect ディレクティブが存在しなくても袖がある場合は必ず shadow 節を指定しないといけない。

また、reflect ディレクティブの存在するファイルにおいては、reflect ディレクティブが書いてあるブロックを対象範囲とするように必ず shadow 節が指示されていなければならない。

以下は実装未対応。

同じファイルに複数の計算オフロードブロック（parallel ディレクティブなど）があり、それぞれに reflect ディレクティブが存在する場合は、全体を present 節と shadow 節を指定した data ディレクティブで囲うことで、parallel ディレクティブごとに shadow 節を指定する手間を省くことができる。

3.5.2 reflect ディレクティブ

ソースコード 5　C 言語の場合

```
#pragma acc reflect(配列名, 配列名, ...)
```

ソースコード 6　Fortran の場合

```
!$acc reflect(配列名, 配列名, ...)
```

reflect ディレクティブは袖通信のタイミングを指示する指示文である。通信の発生するタイミングで記述する。引数は配列名である。複数の配列を指定する場合はカンマ（, ）で区切る。サイズの指定は不要である。

なお、reflect ディレクティブで指示している配列がその箇所で有効となっている shadow 節に含まれていない場合、エラーとなる。

袖が巡回する場合の記述は XMP を参考にする予定。デフォルトでは巡回しない。

斜めの通信がいらない場合の記述は XMP を参考にする予定。デフォルトでは斜めも袖通信する。

3.6 集団通信（袖通信以外）

リダクション（縮約）は OpenACC 標準仕様の reduction 節を流用する。動作としては、どこかの PE に一旦縮約した後に放送すると GPU の挙動を再現できる（命令ある？スカラーに対する All Reduce）。

その他未定。現時点では、ソースコード 7、ソースコード 8 のような記述を想定している。

ソースコード 7　その他の MN-Core 拡張（C 言語）

```
#pragma acc enter data copyin(a[:N][:N],b[:N],c[:N][:N]) \
bcast_divide(b) \
L2(c[16][1]) L1(c[8][1]) mab(c[16][1]) pe(c[4][1])

#pragma acc parallel
        for(count=...) {
                ...
#pragma acc async_data(b)
#pragma acc alltoall(c)
        }
```

ソースコード 8　その他の MN-Core 拡張（Fortran）

```
!$acc enter data copyin(a(:N, :N),b(:N),c(:N, :N) &
!$acc bcast_divide(b) &
!$acc L2(c[16][1]) L1(c[8][1]) mab(c[16][1]) pe(c[4][1])
```

```

!$acc parallel
        do count=...
                ...
!$acc async_data(b)
!$acc alltoall(c)
        end do
!$acc end parallel
```

3.6.1 async_data ディレクティブ

async_data ディレクティブは各 PE の担当計算範囲が決まっている、PE 間で重複配置された配列の一通りのデータを次の計算ステップで使う場合などに、他の PE の担当範囲の更新されたデータを全 PE 間で同期通信するためのディレクティブである。通信としては MPI の Allgather に相当する（送り元と送り先は同じ配列である）。通信の発生するタイミングで記述する。引数は配列名であり、カンマで区切って複数の配列を指定する記述が可能である。なお、async_data ディレクティブで指示する配列は事前に bcast_divide 節で指示されている必要がある。また、上記の説明の通り、送り元にスカラー変数を指定するといった MPI の Allgather と完全に同じ使い方はできず、あくまで繰り返し SIMD 実行される中で、全 PE が保持していて各 PE で別々の要素が計算される配列の同期を目的としたディレクティブであることに注意する。

メモリサイズの都合上、同期する階層を絞れるなら、その範囲を指定できるようにした方が良いかもしれない？ただし、その場合は bcast_devide にも階層の指示が必要となる。

3.6.2 alltoall ディレクティブ

alltoall ディレクティブは全 PE において、処理系によって一意に定められる PE 通し番号と対称となる index 要素を各 PE 間で交換する、行列転置ためのディレクティブである。通信の発生するタイミングで記述する。引数は配列名であり、カンマで区切って複数の配列を指定する記述が可能である。なお、このディレクティブは正方形の 2 次元配列（正方行列）や立方体の 3 次元配列を 1 次元分割して、各 PE に分割配置する場合に限り記述できる。配列の 1 次元あたりの要素数が総 PE 数以下の場合、各 PE は 1 次元配列を持ち、PE 通し番号と同じ index 番号の要素を交換対象する。

配列の 1 次元あたりの要素数が総 PE 数より大きい場合は、1 次元のあたりの要素数を総 PE 数の倍数に制限して記述可能とし、分割する次元は総 PE 数で割った要素数が各 PE に配置されるものとする。つまり、各 PE は 1 次元分割された 2 次元以上の配列を持つことになる。各 PE のどのデータを交換するかは言語処理系で自動的に管理される。

1 次元分割するためだけにいちいち L2、L1、mab、pe を書かないといけないのは面倒なので、省略記法は要検討。3 次元以上の場合はどの座標方向に入れ替えるか指定できた方が良い？ PE をフルに使わなくていいなら、（どっかの PE が休みになるけど）PE 数の倍数に制限しなくても良い？

3.6.3 その他

SIMD 動作であることと、GPU のユースケースを勘案すると、ある PE に集めるとかは基本的にないと思われる（大体の場合ホストに返せば良い）ので、Gather or Scatter ほかは使わない？XMP には一応 gmove があるが元のコードを書き換える必要がある。PE 間放送も不要？というか表現が大変？でも相互作用とかでは必要？少なくとも当面は実装しない方針。

3.7 デバイス固有の指示の明示

構想として、分割の指示等、デバイス固有の指示については device_type 節をその前に書くことで、異なるバージョンの MN-Core のための定義を記述できるようにする。その際、標準仕様では device_type が有効でないディレクティブについても拡張する。

なお、MN-Core 間のバージョンの違いではなく、GPU などのその他アクセラレータとの間で異なる記述を併記するには、その他のアクセラレータのコンパイラ側で独自拡張が未対応であるため、図 6 のように 3.1 節の定義済みマクロで併記するものとする。なお、図中の else にある置き換え後の定義がないマクロは仕様上、何も書かれていない状態に置き換えられるため、SHADOW は削除される。これにより拡張節以外の enter data の指定などを他のアクセラレータと共通化でき、修正箇所を減らせる。

```
1 #ifdef __MNACC__
2 #define SHADOW shadow(array1[1])
3 #else
4 #define SHADOW
5 #endif
6 #pragma acc enter data copyin(array1[ARRAY_SIZE], array2[ARRAY_SIZE], array3[
      ARRAY_SIZE]) SHADOW
```

図 6　定義済みマクロによる拡張ディレクティブの併記例

4　OpenACC から要求される MNCL の仕様

OpenACC/MN は MNCL への変換を想定しているため、ここでは OpenACC 開発の過程で必要が出てきた MNCL の仕様について説明する。ここで説明する仕様はあくまで構想である。

4.1　袖交換の関数

```
halo_comm(配列名，1要素のサイズ，合計次元数，各次元の要素数の配列，各次元の袖領域の幅の 2 次元配列)

// 実際の使用例(3次元配列の場合)
int array3_size[3] = {5, 5, 4};
int array3_shadow[3][2] = {{1, 2}, {1, 2}, {1, 2}};
halo_comm(array3, sizeof(float), 3, array3_size, array3_shadow);
```

halo_comm 関数は MNCL 上における袖交換を行う関数である。この関数はデフォルト分割を前提としている。また、周期的境界条件ではない前提である。

引数のうち、袖領域の情報を格納した配列は 2 次元配列である。

4.2 ある PE のみで処理を変える場合の if 文

```
// 完全にMN-Core の階層構造を表す
if(L2B_id == 数値 && L1B_id == 数値 && MAB_id == 数値 && PE_id == 数値)
```

ある PE のみで異なる処理をする場合、MN-Core は SIMD 動作であるため、if 文を記述する必要がある。

if 文には PE を特定するための情報として、各階層の id を参照している。ここでの id が各階層で完全に別れているのは、MN-Core 上で実際に id を参照する場合に階層ごとの id が取得されるという話に基づいている（未確認）。MNCL 上でハードウェアの情報に基づいて番号を参照するので、全 PE の id は通し番号ではなく、階層の情報も見ている。L2B_id などは MNCL のビルト変数であり、実行時に値が参照できる。

```
#pragma acc parallel present(...
{
#pragma acc loop
// 配列の先頭要素は計算しない
for(i=1; i<8192*2; i++)
        array1[i] = array2[i] + array3[i];
    ...
}
```

図 7　変換前の OpenACC

```
// 1PE あたり 2 回ループを回す
for(i=0; i<2; i++) {
    // 以下の配列の [0]に、元の配列の先頭要素が配置されている
    if(L2B_id == 0 && L1B_id == 0 && MAB_id == 0 && PE_id == 0 && i == 0)
        continue;
    array1[i] = array2[i] + array3[i];
}
```

図 8　変換後の MNCL デバイスコード

図 7 と図 8 は実際にこの if 文が必要となるケースのコード例である。変換前の図 7 では配列の先頭要素は計算しない。これを MNCL デバイスコードに変換すると SIMD 動作であるため、手放しだとすべての PE で同じ動作をしてしまう。

図 8 では if 文を挟むことで、元の配列の先頭要素に当たる要素だけ計算しないようにしている。なお、ループが 2 回回るので、ループ変数も if 文に含めている。なお、この if 文へ正確に変換するには OpenACC 側で以下のことが必要になる。

- デバイス側の各階層の id マップの正確な把握
- デバイス側で配列が実際にどのように展開され、配置されるのかの正確な把握

特に、配列の配置の把握は OpenACC コンパイラ側で対応を把握できるような仕組みを用意する必要がある。また、各階層の id マップについても事前に把握している必要がある。

4.3 PE ローカル変数の使いまわし構文

MNCL の記法だとホストから持ってきた変数はカーネル引数としてグローバルメモリに配置されるが、MN-Core 上のグローバルメモリは DRAM であり、DRAM は無印 MN-Core や MN-Core2 では PE から離れた位置にある。よって、性能を引き出すには可能な限り PE 上のデータを再利用する必要がある。

ここで説明する構文は PE 上のデータを再利用するための記法である。なお、再利用しない変数については通常通りカーネル引数として記述する。以下、複数カーネルで使い回される PE 上の変数を「使いまわし変数」と呼ぶ。

コード例をソースコード 9 に示す。まず、PE ローカル変数の宣言は通常カーネル関数の中にしか記述できないが、使いまわし変数はグローバルスコープで記述する。グローバルスコープで行うのはあくまで宣言であり、メモリの確保は行わない。また、グローバルスコープで宣言するため、C のグローバル変数と同様に名前は一意でなければならない。

使いまわし変数の初期化は__init 修飾子の付いた関数で行う。まず、new でメモリの確保が行われる。new はこの関数以外では記述できない。その後の DRAM からのデータ転送と代入は使い回さない場合の処理と同様である。__init 修飾子の付いた関数が呼び出せるのは 1 回だけである。

通常のカーネルで計算する際は、カーネル引数に記述する必要はなく、C のグローバル変数のようにアクセス可能である。

DRAM へのデータ転送と削除は__final の付いた関数で行う。ここでのデータ転送も使い回さない変数のときと同様である。最後に delete をし、メモリの解放をする。delete はこの関数以外では記述できない。__final の付いた関数が呼び出せるのは 1 回だけである。

初期化と削除の関数に__init、__final という特別な修飾子を付ける理由は、

- 繰り返し呼び出されることを防ぐため
- メモリの確保、削除箇所が一意であることを明示的にするため
- 生存期間、データの流れをわかりやすくするため

である。これらの関数が通常のカーネルとして書かれている場合、繰り返し呼び出せてしまうという問題が発生する。また、変数の生存期間やデータの流れを追いにくくなる。特別な修飾子を用意することでメモリの確保及び削除場所を追いやすく、安全なコードになる。

なお、__init 関数、__final 関数は new、delete する変数に同じ変数が含まれていない限り、別々の使いまわし変数の初期化、削除のために複数記述することができる。

ソースコード 9　PE 上のデータを使い回すコードの例

```
1  // PE 上で複数カーネル共通に使われる配列の宣言
2  __private double array1[COPY_N];
3  __private double array2[COPY_N];
4  __private double array3[COPY_N];
5
6  // 使い回す配列の初期化(PDM or DRAM からデータ転送＋データ代入)
7  __init void enter(__global double* _arg_array1, __global double* _arg_array2,
       __global double* _arg_array3)
8  {
9  new array1, array2, array3;
10
11 bm1 double buf1_bm1[COPY_N * 64];
12 bm2 double buf1_bm2[COPY_N * 512];
13 distribute(buf1_bm1, buf1_bm2, _arg_array1, COPY_N);
14 distribute_pe(array1, buf1_bm1, COPY_N);
15
16 bm1 double buf2_bm1[COPY_N * 64];
17 bm2 double buf2_bm2[COPY_N * 512];
18 distribute(buf2_bm1, buf2_bm2, _arg_array2, COPY_N);
19 distribute_pe(array2, buf2_bm1, COPY_N);
20
21 bm1 double buf3_bm1[COPY_N * 64];
22 bm2 double buf3_bm2[COPY_N * 512];
23 distribute(buf3_bm1, buf3_bm2, _arg_array3, COPY_N);
24 distribute_pe(array3, buf3_bm1, COPY_N);
25 }
26
27 // 普通のカーネル(計算のみ)
28 // この関数は引数が無いが、使い回さないデータがある場合はカーネル引数となる。
29 __kernel void kernel0()
30 {
31 // カーネル引数がある場合は通常通りここにPDM or DRAM to PE の処理が入る
32 array1[0] = array2[0] + array3[0];
33 }
34
35 // 使い回す配列のPDM or DRAM へのデータ転送と削除
36 __final void exit(__global double* _arg_array1, __global double* _arg_array2,
       __global double* _arg_array3)
37 {
38 collect_pe(buf1_bm1, array1, COPY_N);
39 collect(_arg_array1, buf1_bm2, buf1_bm1, COPY_N);
40 collect_pe(buf2_bm1, array2, COPY_N);
41 collect(_arg_array2, buf2_bm2, buf2_bm1, COPY_N);
42 collect_pe(buf3_bm1, array3, COPY_N);
```

```
collect(_arg_array3, buf3_bm2, buf3_bm1, COPY_N);

delete array1, array2, array3;
}
```

5 更新履歴

版	日付	更新内容
0.12	令和 7 年 2 月 27 日	設計書から分離作成。
0.13	令和 7 年 2 月 28 日	最低限必要なことを記述。
0.14	令和 7 年 4 月 8 日	• Fortran 版の記述を追加 • independent 節について言及 • 袖交換の巡回と斜めの通信の有無について言及 • ホスト・デバイス間通信と集団通信の節を分離 • alltoall のところを更新 • MN-Core 固有の指示を記述する際のマクロを使った例を追加 • その他補足を追加
0.15	令和 7 年 4 月 16 日	bcast_divide 節、async_data ディレクティブに関する記述を追加。L2 節、L1 節をそれぞれ lv2 節、lv1 節に変更。
0.16	令和 7 年 6 月 5 日	lv2 節、lv1 節を L2 節、L1 節に差し戻し。alltoall の説明を修正。
0.17	令和 8 年 1 月 23 日	微加筆修正。一部表現を修正。